Lab 2: ROS 2, tf2 and Homogeneous Transformations

Two-Drone Scenario: Transforms, Trajectories and Quaternions

Jiasheng Gu · Student ID: 24020036020

Abstract

This report documents Lab 2 of the MIT VNAV course, which introduces ROS and the tf2 coordinate-transform framework through a two-drone scenario. Because the available virtual machine ran Ubuntu 24.04 (for which ROS 1 Noetic is not supported), the lab was carried out on ROS 2 Jazzy, the matching release for Ubuntu 24.04. ROS 2 Jazzy was installed from the Tsinghua mirror and a colcon workspace was created; the two_drones_pkg package (ported to ament_cmake) was built successfully. In the static scenario the running nodes, topics and launch-file logic were analysed. Two C++ nodes were then completed: frames_publisher_node publishes the time-varying transforms of two aerial vehicles (AV1 on a circle, AV2 on a parabolic arc), and plots_publisher_node looks up transforms to draw the trajectories. Running the nodes verified that AV1's transform is [cos t, sin t, 0] with yaw = t and AV2's is [sin t, 0, cos 2t], and that the /visuals marker array contains both drone meshes and the three trajectory trails. Analytically, the relative trajectory is shown to be a planar curve: its equation is $z = 1 + 2y$ in AV1's frame, and after a change of coordinates it becomes the ellipse $x_p^2/(1/2)^2 + y_p^2/(\sqrt{5}/2)^2 = 1$ with semi-axes 1/2 and $\sqrt{5}/2$. Finally, the orthogonality, identity-rotation and commutativity properties of the quaternion product matrices $\Omega 1$ and $\Omega 2$ are proved. The report includes the complete node source code, the exact commands executed, and a list of every file created or modified.

Keywords: ROS 2; tf2; homogeneous transformation; quaternion; trajectory; ellipse.

1 Introduction

ROS (Robot Operating System) is the middleware that connects the many modules of a robotic system. Its tf2 library keeps track of coordinate frames and the rigid transformations between them, so that sensor data expressed in one frame (e.g. a laser frame) can be transformed into another (e.g. the robot base frame) without manually chaining rotations and translations. Homogeneous 4×4 matrices are the underlying mathematical tool that represents a rotation plus a translation in a single linear map.

In this lab we reproduce, in ROS 2 and RViz2, a scene with two aerial vehicles: AV1 follows a circle while AV2 follows a parabolic arc. We implement the transforms in C++, visualise the trajectories, and then study the trajectory of AV2 relative to AV1 analytically using homogeneous transformations and quaternions.

2 Environment Setup

The virtual machine runs Ubuntu 24.04.3 LTS, so ROS 2 Jazzy Jalisco (the release matched to Ubuntu 24.04) was installed instead of ROS 1 Noetic. The ROS 2 apt repository was added using the Tsinghua mirror (the default packages.ros.org endpoint failed with a certificate error), and the desktop variant plus the colcon build tool were installed:

```
# add the ROS 2 apt repository (Tsinghua mirror)
sudo apt install software-properties-common curl
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
    -o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
    https://mirrors.tuna.tsinghua.edu.cn/ros2/ubuntu noble main" \
    | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

sudo apt update
sudo apt install ros-jazzy-desktop
sudo apt install python3-colcon-common-extensions
```

A colcon workspace was created and the two_drones_pkg package (ported from the lab codebase to ROS 2 ament_cmake) was placed under src/ and built:

```
mkdir -p ~/vnav_ws/src
# copy two_drones_pkg into ~/vnav_ws/src (source files listed in Sec. 10)
cd ~/vnav_ws
source /opt/ros/jazzy/setup.bash
```

```
colcon build --symlink-install
source install/setup.bash
```

The build finished with “Summary: 1 package finished” and produced the two executables frames_publisher_node and plots_publisher_node under install/two_drones_pkg/lib/two_drones_pkg/.

3 Deliverable 1 — Nodes, Topics, Launch Files (10 pts)

The static scenario is started with `ros2 launch two_drones_pkg two_drones.launch.yaml static:=true`. The launch file declares an argument `static` (default `false`) and uses `if`/`unless` conditions to select which nodes run. The launch file is:

```
launch:
    - arg:
        name: "static"
        default: "false"

    - node:
        if: "$(var static)"
        pkg: tf2_ros
        exec: "static_transform_publisher"
        name: "av1broadcaster"
        args: "1 0 0 0 0 0 world av1"

    - node:
        if: "$(var static)"
        pkg: tf2_ros
        exec: "static_transform_publisher"
        name: "av2broadcaster"
        args: "0 0 1 0 0 0 world av2"

    - node:
        unless: "$(var static)"
        pkg: two_drones_pkg
        exec: "frames_publisher_node"
        name: "frames_publisher_node"

    - node:
        pkg: two_drones_pkg
        exec: "plots_publisher_node"
        name: "plots_publisher_node"

    - node:
        pkg: rviz2
        exec: "rviz2"
        name: "rviz2_node"
        args: "-d $(find-pkg-share two_drones_pkg)/config/default.rviz"
```

3.1 Nodes in the static scenario

The following nodes run when static:=true:

Node	Package / executable	Role
av1broadcaster	tf2_ros / static_transform_publisher	Publishes a static world→av1 transform
av2broadcaster	tf2_ros / static_transform_publisher	Publishes a static world→av2 transform
plots_publisher_node	two_drones_pkg	Publishes the drone meshes and trajectories
rviz2_node	rviz2	Visualizes tf frames and markers

3.2 Running the static scenario without ros2 launch

Each node can be started manually with ros2 run in a separate terminal:

```
# terminal 1
ros2 run tf2_ros static_transform_publisher 1 0 0 0 0 0 world av1
# terminal 2
ros2 run tf2_ros static_transform_publisher 0 0 1 0 0 0 world av2
# terminal 3
ros2 run two_drones_pkg plots_publisher_node
# terminal 4
rviz2 -d $(ros2 pkg prefix two_drones_pkg)/share/two_drones_pkg/config/default.rviz
```

3.3 Published / subscribed topics

The nodes exchange data through topics. The relevant publisher/subscriber relationships are:

Node	Publishes	Subscribes
av1broadcaster / av2broadcaster	/tf and /tf_static (world→av1, world→av2)	—
frames_publisher_node	/tf (dynamic av1, av2)	—
plots_publisher_node	/visuals (MarkerArray)	/tf (via tf buffer)
rviz2	—	/tf, /visuals

The av1 and av2 frames are published by av1broadcaster/av2broadcaster in the static case, and by frames_publisher_node in the dynamic case. The topic that makes RViz2 plot the drone meshes is /visuals: plots_publisher_node publishes a visualization_msgs::msg::MarkerArray containing two MESH_RESOURCE markers (the quadrotor.dae mesh) plus the three LINE_STRIP / LINE_LIST trail markers.

3.4 What changes if static:=true is omitted

Omitting static:=true leaves the default `static=false`. In the launch file the two static_transform_publisher nodes are declared with `if: "$(var static)"`, so they are not started. frames_publisher_node is declared with `unless: "$(var static)"`, so it is started. As a result the drones move: frames_publisher_node publishes the time-varying transforms described in the problem formulation instead of fixed poses. In short, `if` runs a node/group only when the condition is true, `unless` runs it only when the condition is false.

4 Deliverable 2 — Publishing Transforms (30 pts)

In frames_publisher_node.cpp three pieces were filled in: (1) a tf2_ros::TransformBroadcaster member, (2) the elapsed time, and (3) the two TransformStamped messages and their publication. The orientation of AV1 uses setRPY(0, 0, time) so that roll = pitch = 0 and yaw = time; this keeps the body y axis tangent to the circular trajectory and the body z axis parallel to the world z axis, as required. One ROS 2 Jazzy adaptation was needed: the legacy four-argument create_timer(this, clock, period, callback) overload was removed in Jazzy, so the node uses the three-argument form create_timer(period, callback). The complete file is:

```
#include <tf2/LinearMath/Quaternion.h>
#include <tf2_ros/transform_broadcaster.h>

#include <chrono>
#include <cmath>
#include <geometry_msgs/msg/transform_stamped.hpp>
#include <memory>
#include <rclcpp/create_timer.hpp>
#include <rclcpp/rclcpp.hpp>
#include <rclcpp/timer.hpp>

class FramesPublisherNode : public rclcpp::Node {
  rclcpp::Time startup_time;
  rclcpp::TimerBase::SharedPtr heartbeat;
  std::unique_ptr<tf2_ros::TransformBroadcaster> br_;

 public:
  FramesPublisherNode() : Node("frames_publisher_node") {
    br_ = std::make_unique<tf2_ros::TransformBroadcaster>(this);
    startup_time = now();
    heartbeat = create_timer(std::chrono::milliseconds(20),
                             std::bind(&FramesPublisherNode::onPublish, this));
    heartbeat->reset();
  }

  void onPublish() {
    double time = (this->now() - startup_time).seconds();

    geometry_msgs::msg::TransformStamped world_T_av1;
    geometry_msgs::msg::TransformStamped world_T_av2;
```

```
    // AV1: origin [cos t, sin t, 0], yaw = t (roll = pitch = 0)
    world_T_av1.header.stamp = this->now();
    world_T_av1.header.frame_id = "world";
    world_T_av1.child_frame_id = "av1";
    world_T_av1.transform.translation.x = std::cos(time);
    world_T_av1.transform.translation.y = std::sin(time);
    world_T_av1.transform.translation.z = 0.0;
    tf2::Quaternion q_av1;
    q_av1.setRPY(0.0, 0.0, time);
    world_T_av1.transform.rotation.x = q_av1.x();
    world_T_av1.transform.rotation.y = q_av1.y();
    world_T_av1.transform.rotation.z = q_av1.z();
    world_T_av1.transform.rotation.w = q_av1.w();

    // AV2: origin [sin t, 0, cos 2t], identity rotation
    world_T_av2.header.stamp = this->now();
    world_T_av2.header.frame_id = "world";
    world_T_av2.child_frame_id = "av2";
    world_T_av2.transform.translation.x = std::sin(time);
    world_T_av2.transform.translation.y = 0.0;
    world_T_av2.transform.translation.z = std::cos(2.0 * time);
    world_T_av2.transform.rotation.w = 1.0;

    br_->sendTransform(world_T_av1);
    br_->sendTransform(world_T_av2);
  }
};

int main(int argc, char** argv) {
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<FramesPublisherNode>());
  rclcpp::shutdown();
  return 0;
}
```

Listing 1: Complete frames_publisher_node.cpp (ROS 2).

5 Deliverable 3 — Looking Up a Transform (30 pts)

In plots_publisher_node.cpp the TrajTrail::update() method fills the transform with the relative pose of dest_frame expressed in ref_frame. lookupTransform(target, source, time) returns source in target, so we call lookupTransform(ref_frame, dest_frame, tf2::TimePointZero). A canTransform guard avoids the timing errors that occur when the transform is polled before it is available. The edited section is:

```
if (parent->tf_buffer_->canTransform(
        ref_frame, dest_frame, tf2::TimePointZero,
        tf2::durationFromSec(0.5))) {
  transform = parent->tf_buffer_->lookupTransform(
      ref_frame, dest_frame, tf2::TimePointZero);
} else {
  return;  // transform not available yet, skip this update
}
```

The three trails are instantiated as TrajTrail("world", "av1"), TrajTrail("world", "av2") and TrajTrail("av1", "av2"), which draw AV1 in the world frame (circle), AV2 in the world frame (parabola) and AV2 in AV1's frame (the relative, dashed trajectory). The complete file is:

```
#include <tf2/LinearMath/Quaternion.h>
#include <tf2/exceptions.h>
#include <tf2/time.h>
#include <tf2_ros/buffer_interface.h>
#include <tf2_ros/transform_broadcaster.h>

#include <chrono>
#include <geometry_msgs/msg/transform_stamped.hpp>
#include <iostream>
#include <list>
#include <memory>
#include <rclcpp/create_timer.hpp>
#include <rclcpp/duration.hpp>
#include <rclcpp/publisher.hpp>
#include <rclcpp/rclcpp.hpp>
#include <rclcpp/timer.hpp>
#include <visualization_msgs/msg/marker.hpp>
#include <visualization_msgs/msg/marker_array.hpp>
#include "tf2_ros/buffer.h"
```

```
#include "tf2_ros/transform_listener.h"

class PlotsPublisherNode : public rclcpp::Node {
  rclcpp::Time startup_time;
  rclcpp::TimerBase::SharedPtr heartbeat;
  rclcpp::Publisher<visualization_msgs::msg::MarkerArray>::SharedPtr markers_pub;
  std::shared_ptr<tf2_ros::TransformListener> tf_listener_{nullptr};
  std::unique_ptr<tf2_ros::Buffer> tf_buffer_;
  int num_trails;

  class TrajTrail {
    PlotsPublisherNode* parent;
    static int id;
    std::list<geometry_msgs::msg::Point> poses;
    std::string ref_frame, dest_frame;
    size_t buffer_size;
    std::string ns;
    visualization_msgs::msg::Marker marker_out;

    void update() {
      geometry_msgs::msg::TransformStamped transform;
      try {
        if (parent->tf_buffer_->canTransform(
                ref_frame, dest_frame, tf2::TimePointZero,
                tf2::durationFromSec(0.5))) {
          transform = parent->tf_buffer_->lookupTransform(
              ref_frame, dest_frame, tf2::TimePointZero);
        } else {
          return;  // transform not available yet
        }
        while (poses.size() >= buffer_size) poses.pop_front();
        geometry_msgs::msg::Point tmp;
        tmp.x = transform.transform.translation.x;
        tmp.y = transform.transform.translation.y;
        tmp.z = transform.transform.translation.z;
        poses.push_back(tmp);
      } catch (const tf2::TransformException& ex) {
        RCLCPP_ERROR_STREAM(parent->get_logger(),
                            "transform lookup failed: " << ex.what());
      }
    }

   public:
    TrajTrail() : parent(NULL){};
    TrajTrail(PlotsPublisherNode* parent_, const std::string& ref_frame_,
              const std::string& dest_frame_, int buffer = 160)
        : parent(parent_), ref_frame(ref_frame_), dest_frame(dest_frame_),
          buffer_size(buffer) {
      marker_out.header.frame_id = ref_frame;
      marker_out.ns = "trails";
      marker_out.id = parent->num_trails++;
      marker_out.type = visualization_msgs::msg::Marker::LINE_STRIP;
      marker_out.action = visualization_msgs::msg::Marker::ADD;
      marker_out.color.a = 0.8;
      marker_out.scale.x = 0.02;
      marker_out.lifetime = rclcpp::Duration::from_seconds(1.0);
    }

    void setColor(float r_, float g_, float b_) {
      marker_out.color.r = r_; marker_out.color.g = g_; marker_out.color.b = b_;
    }
    void setNamespace(const std::string& ns_) { marker_out.ns = ns_; }
    void setDashed() { marker_out.type = visualization_msgs::msg::Marker::LINE_LIST; }

    visualization_msgs::msg::Marker getMarker() {
      update();
      marker_out.header.stamp = parent->now();
      marker_out.points.clear();
      for (auto& p : poses) marker_out.points.push_back(p);
      if (marker_out.type == visualization_msgs::msg::Marker::LINE_LIST &&
          poses.size() % 2)
        marker_out.points.resize(poses.size() - 1);
      return marker_out;
    }
  };

  TrajTrail av1trail, av2trail, av2trail_rel;
```

```
 public:
  PlotsPublisherNode() : Node("plots_publisher_node"), num_trails(0) {
    RCLCPP_INFO_STREAM(get_logger(), "Started plot node");
    tf_buffer_ = std::make_unique<tf2_ros::Buffer>(this->get_clock());
    tf_listener_ = std::make_shared<tf2_ros::TransformListener>(*tf_buffer_);
    startup_time = now();
    markers_pub = create_publisher<visualization_msgs::msg::MarkerArray>("visuals", 0);
    heartbeat = create_timer(std::chrono::milliseconds(20),
                             std::bind(&PlotsPublisherNode::onPublish, this));
    heartbeat->reset();

    av1trail = TrajTrail(this, "world", "av1", 300);
    av1trail.setColor(.25, .52, 1);
    av1trail.setNamespace("Trail av1-world");
    av2trail = TrajTrail(this, "world", "av2", 300);
    av2trail.setColor(.8, .4, .26);
    av2trail.setNamespace("Trail av2-world");
    av2trail_rel = TrajTrail(this, "av1", "av2", 160);
    av2trail_rel.setDashed();
    av2trail_rel.setColor(.8, .4, .26);
    av2trail_rel.setNamespace("Trail av2-av1");
  }

  void onPublish() {
    visualization_msgs::msg::MarkerArray visuals;
    visuals.markers.resize(2);
    visualization_msgs::msg::Marker& av1(visuals.markers[0]);
    visualization_msgs::msg::Marker& av2(visuals.markers[1]);
    av1.header.frame_id = "av1";
    av1.ns = "AVs";
    av1.id = 0;
    av1.header.stamp = now();
    av1.type = visualization_msgs::msg::Marker::MESH_RESOURCE;
    av1.mesh_resource = "package://two_drones_pkg/mesh/quadrotor.dae";
    av1.action = visualization_msgs::msg::Marker::ADD;
    av1.pose.orientation.w = 1.0;
    av1.scale.x = av1.scale.y = av1.scale.z = av1.color.a = 1.0;
    av1.color.r = .25; av1.color.g = .52; av1.color.b = 1;
    av1.lifetime = rclcpp::Duration::from_seconds(1.0);
    av2 = av1;
    av2.header.frame_id = "av2";
    av2.ns = "AVs";
    av2.id = 1;
    av2.color.r = .8; av2.color.g = .4; av2.color.b = .26;

    visuals.markers.push_back(av1trail.getMarker());
    visuals.markers.push_back(av2trail.getMarker());
    visuals.markers.push_back(av2trail_rel.getMarker());
    markers_pub->publish(visuals);
  }
};

int main(int argc, char** argv) {
  rclcpp::init(argc, argv);
  rclcpp::spin(std::make_shared<PlotsPublisherNode>());
  return 0;
}
```

Listing 2: Complete plots_publisher_node.cpp (ROS 2).

6 Deliverable 4 — Mathematical Derivations (25 pts)

6.1 AV2's world trajectory is a parabola

The world-frame positions are:

$$o_1^w = [\cos t,\ \sin t,\ 0]^T, \qquad o_2^w = [\sin t,\ 0,\ \cos 2t]^T$$

For AV2 the y coordinate is identically zero, so the trajectory lies in the x-z plane. Writing x = sin t and using cos 2t = 1 − 2 sin²t gives:

$$z = \cos 2t = 1 - 2\sin^2 t = 1 - 2x^2$$

which is a downward-opening parabola in the x-z plane.

6.2 Position of AV2 in AV1's body frame

AV1's orientation satisfies roll = pitch = 0, yaw = t, so its rotation relative to the world is the elementary z-rotation:

```
R1^w = Rz(t) =

[ cos t   -sin t   0 ]
[ sin t   cos t    0 ]
[ 0       0        1 ]
```

The homogeneous transform from world to AV1 is T1^w = [R1^w | o1^w; 0 1], with o1^w = [cos t, sin t, 0]^T. The inverse maps a world point into AV1's frame: o2¹ = $(R1^w)^T$ (o2^w − o1^w). Computing o2^w − o1^w = [sin t − cos t, −sin t, cos 2t]^T and applying $(R1^w)^T$ = Rz(−t) yields:

$$o_2^1(t) = [\sin t\cos t - 1, \ \ -\sin^2 t, \ \cos 2t]^T$$

6.3 The relative trajectory is planar

The z and y components of o2¹ satisfy z = cos 2t = 1 − 2 sin²t = 1 + 2(−sin²t) = 1 + 2y. Hence:

$$z = 1 + 2y \quad \Rightarrow \quad \Pi: \ z - 2y - 1 = 0$$

The trajectory lies in the plane Π with normal n = (0, −2, 1), independent of x.

6.4 A 2D frame centred at the origin

The centre of the curve is p = (−1, −1/2, 0) (given). Subtracting it from o2¹ and using sin 2t = 2 sin t cos t gives:

$$o_2^p(t) = \left[\tfrac{1}{2}\sin 2t, \ \tfrac{\sqrt{5}}{2}\cos 2t, \ 0\right]^T$$

Choose the in-plane axes xp along x1 = (1,0,0) and yp along (0,1,2)/√5 (the in-plane direction orthogonal to xp), with zp along the normal (0,−2,1)/√5. Projecting, the z component vanishes and the 2D coordinates are xp = (1/2) sin 2t and yp = (√5/2) cos 2t.

6.5 The trajectory is an ellipse

Since sin²2t + cos²2t = 1:

$$\frac{x_p^2}{(1/2)^2} + \frac{y_p^2}{(\sqrt{5}/2)^2} = 1$$

Therefore the curve is an ellipse with semi-axes:

$$a = \tfrac{1}{2}, \qquad b = \tfrac{\sqrt{5}}{2}$$

The semi-major axis is √5/2 ≈ 1.118 (along yp) and the semi-minor axis is 1/2 = 0.5 (along xp). The ellipse is centred at p = (−1, −1/2, 0) in AV1's frame and lies in the plane Π: z − 2y − 1 = 0.

7 Deliverable 5 — Properties of Quaternions (5 pts)

The two product matrices are defined as:

```
Omega1(q) = [ q4    -q3   q2    q1 ]    Omega2(q) = [ q4    q3    -q2   q1 ]
            [ q3    q4    -q1   q2 ]                [ -q3   q4    q1    q2 ]
            [ -q2   q1    q4    q3 ]                [ q2    -q1   q4    q3 ]
            [ -q1   -q2   -q3   q4 ]                [ -q1   -q2   -q3   q4 ]
```

7.1 Orthogonality of Ω1(q) and Ω2(q)

For a unit quaternion (q1²+q2²+q3²+q4² = 1), the four columns of Ω1(q) — namely $[q4,q3,-q2,-q1]^T$, $[-q3,q4,q1,-q2]^T$, $[q2,-q1,q4,-q3]^T$ and $[q1,q2,q3,q4]^T$ — are pairwise orthogonal and each has squared norm equal to |q|² = 1. Therefore $\Omega1(q)^T\Omega1(q) = \Omega1(q)\Omega1(q)^T = I4$, i.e. Ω1(q) is orthogonal; the same argument applies to Ω2(q). Intuitively, quaternion multiplication preserves the norm (‖qa ⊗ qb‖ =

‖qa‖‖qb‖), so the matrix representing left (or right) multiplication by a unit quaternion must preserve the Euclidean norm, which is exactly the defining property of an orthogonal matrix.

7.2 $\Omega1(q)^T q = \Omega2(q)^T q = [0,0,0,1]^T$

$\Omega1(q)^T q$ is the product of Ω1(q)'s columns with q. The first three entries vanish by the same pairwise cancellation above (e.g. q4·q1 + q3·q2 − q2·q3 − q1·q4 = 0), while the last entry is $q1^2+q2^2+q3^2+q4^2 = |q|^2 = 1$. Hence $\Omega1(q)^T q = [0,0,0,1]^T$, and identically for Ω2(q). Geometrically this says that left-multiplying q by its conjugate q* yields the identity rotation [0,0,0,1].

7.3 Commutativity Ω1(x)Ω2(y) = Ω2(y)Ω1(x)

For any z, Ω1(x)Ω2(y) z = x ⊗ (z ⊗ y) while Ω2(y)Ω1(x) z = (x ⊗ z) ⊗ y. Quaternion multiplication is associative, so x ⊗ (z ⊗ y) = (x ⊗ z) ⊗ y, and therefore the two linear maps coincide: Ω1(x)Ω2(y) = Ω2(y)Ω1(x). The same associativity argument with y replaced by its conjugate y* gives $\Omega1(x)\Omega2(y)^T = \Omega2(y)^T\Omega1(x)$, since $\Omega2(y)^T$ represents right-multiplication by y*.

8 Deliverable 6 — Intrinsic vs Extrinsic Rotations (Optional, 20 pts)

The rotation sequence is R0 (90° about x), R1 (180° about y), R2 (−30° about x). In the extrinsic case the rotations are applied about the fixed world axes in the order R0, R1, R2; in the intrinsic case the same rotations are applied about the moving body axes in the reverse order R2, R1, R0. The claim is that both lead to the same final orientation.

Two facts make the proof immediate:

Fact 1 (pre-multiplication = rotation about a global axis). If the current orientation is R and we rotate by R_e about a fixed world axis, the new orientation is R' = R_e · R, i.e. we pre-multiply by R_e.

Fact 2 (post-multiplication = rotation about a local axis). If the current orientation is R and we rotate by R_e about the current body axis e, the new orientation is R' = R · R_e, i.e. we post-multiply by R_e. To see this, note that the body axis e points along R·e in the world frame, so the world-frame rotation about that axis is the similarity transform R · R_e · R^T; applying it to R gives (R · R_e · R^T) · R = R · R_e.

Applying these facts to the two sequences, starting from the identity orientation I:

```
Extrinsic (global axes, order R0 -> R1 -> R2):
    R_ext = R2 * (R1 * (R0 * I)) = R2 * R1 * R0

Intrinsic (body axes, reverse order R2 -> R1 -> R0):
    R_int = ((I * R2) * R1) * R0 = R2 * R1 * R0
```

Hence R_ext = R_int = R2 · R1 · R0, so the final orientation is identical. The argument never uses the specific axes, angles or number of rotations, so the property holds in general: an extrinsic sequence R0, R1, …, Rn (pre-multiplications) equals the intrinsic sequence Rn, …, R1, R0 (post-multiplications), because both produce the same matrix product Rn ⋯ R1 R0.

9 Practical Run and Verification

After building, both nodes were launched in the workspace and their output was verified. The topics /tf, /tf_static and /visuals appeared, with /tf carrying the world→av1 and world→av2 transforms and /visuals carrying the marker array.

```
source /opt/ros/jazzy/setup.bash
source ~/vnav_ws/install/setup.bash
ros2 run two_drones_pkg frames_publisher_node &
ros2 run two_drones_pkg plots_publisher_node &
ros2 topic list          # -> /tf, /tf_static, /visuals
ros2 topic info /tf      # -> Publisher count: 1
ros2 topic info /visuals # -> Publisher count: 1
ros2 topic echo /tf      # sample below
```

A sample of the /tf topic confirms the transforms are exactly as specified. For AV1 the translation is [cos t, sin t, 0] and the quaternion is the yaw = t rotation (roll = pitch = 0); for AV2 the translation is [sin t, 0, cos 2t] with identity rotation:

```
# world -> av1
translation: { x: -0.9206, y: 0.3905, z: 0.0 }   # = [cos t, sin t, 0]
rotation:    { x: 0.0, y: 0.0, z: 0.9799, w: 0.1993 }  # yaw = t

# world -> av2
translation: { x: 0.3905, y: 0.0, z: 0.6950 }    # = [sin t, 0, cos 2t]
rotation:    { x: 0.0, y: 0.0, z: 0.0, w: 1.0 }  # identity
```

Listing 3: Verified /tf output.

The /visuals marker array was also inspected: it contains the two MESH_RESOURCE markers (AVs namespace) and the three trail markers (Trail av1-world, Trail av2-world, Trail av2-av1), with hundreds of accumulated points, and no “transform lookup failed” errors appeared in the node log — confirming that the canTransform / lookupTransform logic in Deliverable 3 works correctly.

Because RViz2 is a graphical application, it cannot be shown over a plain SSH session; it is started by running the launch file directly in the virtual machine's desktop:

```
ros2 launch two_drones_pkg two_drones.launch.yaml
```

10 Files Created and Modified

The package lives at ~/vnav_ws/src/two_drones_pkg (with compiled artefacts under ~/vnav_ws/build, ~/vnav_ws/install and ~/vnav_ws/log). The complete list of files is given in Table 1.

Path	Type	Purpose
src/two_drones_pkg/CMakeLists.txt	created	ament_cmake build description (Jazzy-adapted)
src/two_drones_pkg/package.xml	created	Package manifest (build_type ament_cmake)
src/two_drones_pkg/src/frames_publisher_node.cpp	completed	Publishes world→av1/av2 transforms (D2)
src/two_drones_pkg/src/plots_publisher_node.cpp	completed	Looks up transforms, publishes trails (D3)
src/two_drones_pkg/launch/two_drones.launch.yaml	created	Launch file (static + dynamic scenarios)
src/two_drones_pkg/config/default.rviz	kept	RViz2 configuration (fixed frame world)
src/two_drones_pkg/mesh/quadrotor.dae	kept	Quadrotor 3D mesh resource
build/	generated	colcon build intermediate files
install/	generated	colcon install tree (setup.bash, executables, mesh)
install/.../lib/two_drones_pkg/*	generated	frames_publisher_node, plots_publisher_node executables
log/	generated	colcon build logs

Table 1: Files created, completed or generated during Lab 2.

11 Conclusion

Lab 2 introduced ROS, the tf2 transform framework and homogeneous transformations through a two-drone scenario. A ROS 2 Jazzy environment and a colcon workspace were set up, and the two_drones_pkg package was built and run. The ROS nodes, topics and launch-file logic of the static scene were analysed; the two C++ nodes were completed so that AV1 follows a circle, AV2 follows a parabola, and the relative trajectory of AV2 in AV1's frame is drawn. The analytical study shows that this relative trajectory is an ellipse lying in the plane $z = 1 + 2y$ of AV1's frame, with semi-axes 1/2 and $\sqrt{5}/2$. The quaternion product matrices $\Omega 1$ and $\Omega 2$ were shown to be orthogonal, to map a unit quaternion to the identity rotation, and to commute, reflecting the associativity of quaternion multiplication. The practical run confirmed that both nodes publish correct transforms and markers, completing all six deliverables.

References

[1] MIT VNAV Lab 2 exercises, 2023. https://vnav.mit.edu/labs_2023/lab2/exercises.html

[2] Lab 2: ROS (course notes). https://spurious-cornflower-507.notion.site/Lab2-ROS-105e9f90e72480519605ed793e6662dc

[3] MIT-SPARK VNAV-labs source code. https://github.com/MIT-SPARK/VNAV-labs
[4] ROS 2 Jazzy documentation. https://docs.ros.org/en/jazzy/
[5] tf2 tutorials. https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Tf2/Tf2-Main.html